

AlIDocs 阿里云 ECS 部署指南

本文基于仓库内现有生产配置（`docker-compose.prod.yml`、`scripts/server-bootstrap.sh`）整理，适用于在单台阿里云 ECS 上以 Docker Compose 运行完整 AlIDocs 栈。

代码同步方式（二选一）：

- 1. 本地上传（默认文档流程）：开发机 `rsync` 或压缩包上传至 `/opt/alldocs`，不依赖 GitHub 存放代码。
- 2. GitHub Actions 自动部署（可选）：推送 `main` 分支后，`.github/workflows/deploy.yml` 经 SSH `rsync` 同步代码并执行 `scripts/remote_compose_deploy.sh`。需在仓库 Secrets 配置 `ECS_HOST`、`ECS_USER`、`ECS_SSH_KEY`；`.env` 与 `models/` 仍只保留在服务器本地，不会被 CI 覆盖。

一、部署架构概览

生产环境通过 `docker-compose.prod.yml` 拉起以下容器：

服务	说明	是否对外暴露
<code>frontend</code> (Nginx)	静态前端 + 反向代理 <code>/api</code> 、 <code>/ws</code>	是（默认 80 端口）
<code>api</code>	FastAPI 主服务	否（经 Nginx 代理）
<code>inference</code>	BGE 向量 / Rerank 推理（减轻 <code>api/worker</code> 内存）	否
<code>worker-ingestion</code>	文档入库 Celery Worker	否
<code>worker-maintenance</code>	删除/清理 Celery Worker	否
<code>postgres</code> / <code>redis</code> / <code>qdrant</code> / <code>elasticsearch</code> / <code>minio</code>	数据存储	否

对外只开放 前端 Nginx 的 80 端口；API 通过 Nginx 的 `/api/` 和 WebSocket `/ws/` 转发，无需单独暴露 8000。Nginx 默认 `client_max_body_size 100m`（见 `frontend/nginx.conf`），大 PDF 上传需确保未在宿主机反向代理层进一步限缩。

二、资源规划（阿里云 ECS）

2.1 推荐规格

场景	CPU	内存	系统盘	说明
最低可用（CPU 推理）	4 核	16 GB	100 GB ESSD	可跑通；大 PDF 入库可能较慢
推荐生产	8 核	32 GB	200 GB ESSD	含 Elasticsearch、Whisper small、BGE 模型
高并发 / 大文档	16 核 +	64 GB+	500 GB+	可调大 CELERY_INGESTION_CONCURRENCY

语音模型默认 `WHISPER_MODEL=small`（约数 GB 内存）。`large-v3` 需 10 GB+ 专用内存，容器内易 OOM（exit 137）。

2.2 操作系统

- 推荐：Ubuntu 22.04 LTS 或 Alibaba Cloud Linux 3
- 需能安装 Docker Engine + Compose 插件

2.3 安全组入方向规则

端口	协议	来源	用途
22	TCP	你的办公 IP（勿对全网开放）	SSH 运维
80	TCP	<code>0.0.0.0/0</code>	HTTP 访问应用
443	TCP	<code>0.0.0.0/0</code>	HTTPS（若配置证书）

不要对公网开放：5432、6379、6333、9200、9000、8000、8100。

2.4 域名与备案（可选）

- 使用中国大陆 ECS + 公网域名访问，需完成 ICP 备案
- 可在阿里云购买域名，解析 A 记录到 ECS 公网 IP
- HTTPS 见本文「第六节」

三、首次部署

3.1 创建并登录 ECS

1. 阿里云控制台 → 云服务器 ECS → 创建实例（按上表选规格）
2. 登录方式：SSH 密钥对（推荐）或密码
3. 本地连接：

```
ssh -i ~/.ssh/your-key.pem root@<ECS公网IP>
```

3.2 上传代码到服务器

在开发机（已有一份 AllDocs 源码的目录）执行，将项目同步到 ECS 的 `/opt/alldocs`。以下命令将 `<ECS公网IP>`、SSH 密钥路径替换为实际值。

方式 A： `rsync`（推荐，适合首次部署与日常更新）

```
# 在开发机，进入项目根目录
cd /path/to/AllDocs

ssh -i ~/.ssh/your-key.pem root@<ECS公网IP> "mkdir -p /opt/alldocs"

rsync -avz --delete \
  --exclude '.git/' \
  --exclude '.env' \
  --exclude 'node_modules/' \
  --exclude 'frontend/node_modules/' \
  --exclude 'models/' \
  -e "ssh -i ~/.ssh/your-key.pem" \
  ./ root@<ECS公网IP>:/opt/alldocs/
```

- `--exclude '.env'`：避免覆盖服务器上已配置的密钥
- `--exclude 'models/'`：保留服务器上 bootstrap 下载的语音与 Embedding 模型
- `--delete`：删除服务器上已移除的文件，使目录与本地一致（不会删除上述排除项）

方式 B：压缩包（无法使用 `rsync` 时）

开发机打包并上传：

```
cd /path/to/AllDocs
tar czf /tmp/alldocs-src.tar.gz \
  --exclude='.git' \
  --exclude='node_modules' \
  --exclude='frontend/node_modules' \
  --exclude='.env' \
  --exclude='models' \
  .

scp -i ~/.ssh/your-key.pem /tmp/alldocs-src.tar.gz root@<ECS公网IP>:/tmp/
```

ECS 上解压：

```
mkdir -p /opt/alldocs
tar xzf /tmp/alldocs-src.tar.gz -C /opt/alldocs
```

方式 C：经阿里云 OSS 中转（可选）

1. 开发机将上述 `alldocs-src.tar.gz` 上传到 OSS Bucket
2. ECS 安装 `ossutil` 或控制台生成临时下载链接，在服务器执行：

```
mkdir -p /opt/alldocs
wget -O /tmp/alldocs-src.tar.gz '<OSS签名URL>'
tar xzf /tmp/alldocs-src.tar.gz -C /opt/alldocs
```

建议将项目固定在 `/opt/alldocs`，后续更新与备份路径统一。

3.3 运行初始化脚本

项目提供一键初始化脚本，会完成：

- 安装 Docker + Compose（含国内镜像加速）
- 从 `.env.example` 生成 `.env`
- 下载 Piper 语音模型、BGE Embedding/Rerank 模型（ModelScope，适合国内网络）

```
cd /opt/alldocs
```

```
# 第一步：安装 Docker、下载模型、创建 .env
```

```
bash scripts/server-bootstrap.sh
```

```
# 第二步：自动生成数据库 / MinIO 强密码
```

```
bash scripts/server-bootstrap.sh --generate-secrets
```

`--generate-secrets` 会写入：

- `POSTGRES_PASSWORD`
- `POSTGRES_URL` / `POSTGRES_URL_SYNC` (主机名 `postgres`)
- `MINIO_ACCESS_KEY` / `MINIO_SECRET_KEY`

重要： `.env` 含密钥，仅保存在服务器本地，请自行备份（如加密存到密码管理器）。上传代码时务必排除 `.env`，避免被本地空模板覆盖。

3.4 编辑 `.env`（必做）

```
vim /opt/alldocs/.env
```

(1) LLM API（必填）

方案 A：DeepSeek（默认示例）

```
LLM_API_BASE_URL=https://api.deepseek.com/v1
```

```
LLM_API_KEY=sk-xxxxxxx
```

```
LLM_MODEL=deepseek-chat
```

方案 B：阿里云百炼 DashScope（OpenAI 兼容）

```
LLM_API_BASE_URL=https://dashscope.aliyuncs.com/compatible-mode/v1
```

```
LLM_API_KEY=sk-xxxxxxx
```

```
LLM_MODEL=qwen-plus
```

在 [百炼控制台](#) 创建 API Key。

(2) Docker 内网服务地址（必填）

容器间通信须使用 **Compose 服务名**，不能用 `localhost`：

```
REDIS_URL=redis://redis:6379/0
QDRANT_URL=http://qdrant:6333
ELASTICSEARCH_URL=http://elasticsearch:9200
MINIO_ENDPOINT=minio:9000
MINIO_SECURE=false

# 生产 compose 默认启动 inference 服务，务必配置：
INFERENCE_URL=http://inference:8100
```

`server-bootstrap.sh` 已把 `POSTGRES_URL` 改为 `postgres` 主机名；其余几项需手动改。

(3) 模型路径 (bootstrap 后通常已写好)

```
EMBEDDING_MODEL=/app/models/modelscope/BAAI/bge-m3
RERANK_MODEL=/app/models/modelscope/BAAI/bge-reranker-v2-m3
PIPER_MODEL_DIR=/app/models/piper
```

(4) 生产调优建议

```
# 语音：避免 OOM
WHISPER_MODEL=small
WHISPER_DEVICE=cpu
WHISPER_COMPUTE_TYPE=int8

# Elasticsearch 内存（机器内存紧张时可保持 512m）
ES_JAVA_OPTS=-Xms512m -Xmx512m

# Worker 并发（按 CPU/内存调整）
CELERY_INGESTION_CONCURRENCY=1
CELERY_MAINTENANCE_CONCURRENCY=2

# 对外端口（默认 80）
FRONTEND_PORT=80
```

(5) 可选：入库 VLM（解析 PDF 内嵌图）

```
INGEST_CAPTION_ENABLED=true
INGEST_CAPTION_API_BASE_URL=https://dashscope.aliyuncs.com/compatible-mode/v1
INGEST_CAPTION_API_KEY=sk-xxxxxxx
INGEST_CAPTION_MODEL=qwen-vl-plus
```

未配置时回退到 `LLM_*` 凭据。

(6) 用户认证 (生产必配)

生产环境**不要**使用 `AUTH_DISABLED=true`。至少配置 JWT 密钥与首个管理员账号：

```
# 强随机字符串, 可用: openssl rand -hex 32
JWT_SECRET=your-production-jwt-secret
JWT_ACCESS_TTL_MINUTES=30
JWT_REFRESH_TTL_DAYS=14
AUTH_DISABLED=false

BOOTSTRAP_ADMIN_EMAIL=admin@yourcompany.com
BOOTSTRAP_ADMIN_PASSWORD=your-strong-admin-password
```

API 首次启动时会自动创建上述 Admin (若该邮箱尚未注册)。也可在容器内手动创建：

```
docker compose -f docker-compose.prod.yml exec api \
  python -m app.cli create-admin --email admin@yourcompany.com --password your-strong-p
```

手机验证码 (阿里云 SMS)

```
SMS_PROVIDER=aliyun
SMS_OTP_TTL_SECONDS=300
SMS_OTP_RESEND_SECONDS=60
SMS_OTP_MAX_ATTEMPTS=5

ALIYUN_SMS_ACCESS_KEY_ID=your-access-key-id
ALIYUN_SMS_ACCESS_KEY_SECRET=your-access-key-secret
ALIYUN_SMS_SIGN_NAME=你的短信签名
ALIYUN_SMS_TEMPLATE_CODE=SMS_xxxxxx
ALIYUN_SMS_REGION=cn-hangzhou
ALIYUN_SMS_ENDPOINT=https://dysmsapi.aliyuncs.com
```

- 开发调试可设 `SMS_PROVIDER=console`，验证码会写入 API 日志 (生产勿用)
- 短信模板变量须包含 `code`，与系统发送的 JSON `{"code":"123456"}` 一致
- 在 [阿里云短信控制台](#) 申请签名与模板

邮箱注册验证码 (SMTP)

```
EMAIL_PROVIDER=smtp
EMAIL_OTP_TTL_SECONDS=300
EMAIL_OTP_RESEND_SECONDS=60
EMAIL_OTP_MAX_ATTEMPTS=5

SMTP_HOST=smtp.example.com
SMTP_PORT=587
SMTP_USER=your-smtp-user
SMTP_PASSWORD=your-smtp-password
SMTP_FROM=noreply@example.com
SMTP_USE_TLS=true
```

- 开发调试可设 `EMAIL_PROVIDER=console`，验证码会写入 API 日志（生产勿用）
- 阿里云邮件推送、腾讯企业邮等均可通过 SMTP 接入；`SMTP_FROM` 需与发信域名/账号一致

微信扫码登录（可选）

1. 在 [微信开放平台](#) 创建网站应用，获取 AppID / AppSecret
2. 授权回调域填写你的公网域名（须与 Nginx 对外域名一致）
3. 配置：

```
WECHAT_APP_ID=wxXXXXXXXXX
WECHAT_APP_SECRET=xxxxxxxxx
WECHAT_REDIRECT_URI=https://docs.example.com/api/v1/auth/wechat/callback
AUTH_FRONTEND_CALLBACK_URL=https://docs.example.com/auth/callback
```

- `WECHAT_REDIRECT_URI` 指向 API 的 OAuth 回调（经 Nginx 转发 `/api/`）
- `AUTH_FRONTEND_CALLBACK_URL` 指向 前端 页面，用于登录完成后写入 token
- 绑定微信时前端会跳转到 `/api/v1/auth/wechat/bind/authorize?token=...`

权限说明

能力	Admin	普通用户
查看文档、对话、语音	✓	✓
控制各文档是否参与 RAG（ <code>chat_enabled</code> ）	✓	✗
上传 / 重新索引 / 删除 / 系统设置	✓	✗
用户管理、审计日志	✓	✗
账号绑定 / 解绑	✓	✓

问答检索范围：所有 `ready` 且 `chat_enabled=true` 的文档（Admin 在侧边栏切换开关）。

修改 `.env` 中的认证相关变量后，需重启 API：

```
docker compose -f docker-compose.prod.yml restart api
```

3.5 启动生产栈

```
cd /opt/alldocs
bash scripts/server-bootstrap.sh --start
```

或等价命令：

```
bash scripts/pull_docker_images.sh    # 经国内镜像站预拉基础镜像
docker compose -f docker-compose.prod.yml up -d --build
```

首次构建通常需 20—40 分钟（拉镜像、编译 backend、构建 Elasticsearch IK 插件、前端 npm build）。

3.6 验证部署

```
# 查看容器状态
docker compose -f docker-compose.prod.yml ps

# API 健康检查（容器内）
docker compose -f docker-compose.prod.yml exec -T api \
  python -c "import urllib.request; urllib.request.urlopen('http://localhost:8000/health')"

# 查看日志
docker compose -f docker-compose.prod.yml logs -f api worker-ingestion worker-maintenance
```

浏览器访问：

地址	说明
<code>http://<ECS公网IP>/</code>	前端
<code>http://<ECS公网IP>/api/v1/...</code>	API（经 Nginx）
<code>http://<ECS公网IP>/api/v1/settings</code>	运行时配置

上传一份测试 PDF，确认状态从 `pending` → `processing` → `ready`，再试问答。

四、版本更新与重新部署

有新版本时，在开发机重新同步代码，再 SSH 登录 ECS 重建容器。

4.1 同步代码（开发机）

```
cd /path/to/AllDocs

rsync -avz --delete \
  --exclude '.git/' \
  --exclude '.env' \
  --exclude 'node_modules/' \
  --exclude 'frontend/node_modules/' \
  --exclude 'models/' \
  -e "ssh -i ~/.ssh/your-key.pem" \
  ./ root@<ECS公网IP>:/opt/alldocs/
```

无法使用 `rsync` 时，按第三节「方式 B / C」打包上传，解压覆盖 `/opt/alldocs`（同样不要覆盖 `.env` 与 `models/`）。

4.2 重建并启动（ECS）

```
cd /opt/alldocs

bash scripts/pull_docker_images.sh
docker compose -f docker-compose.prod.yml up -d --build
```

若仅修改了后端代码、无需重建前端，可缩小重建范围：

```
docker compose -f docker-compose.prod.yml up -d --build \
  api worker-ingestion worker-maintenance inference
```

仅修改了前端时：

```
docker compose -f docker-compose.prod.yml up -d --build frontend
```

更新后验证：

```
docker compose -f docker-compose.prod.yml ps
docker compose -f docker-compose.prod.yml exec -T api \
  python -c "import urllib.request; urllib.request.urlopen('http://localhost:8000/health')
```

五、HTTPS 与域名（推荐）

生产 compose 的 Nginx 仅监听 80。常见做法：

方案 A：宿主机 Nginx + Let's Encrypt（推荐）

```
apt-get install -y nginx certbot python3-certbot-nginx

# 先将域名 A 记录指向 ECS IP
certbot --nginx -d docs.example.com
```

在宿主机 Nginx 中反向代理到 `127.0.0.1:80`（或改 `FRONTEND_PORT=8080` 让容器监听 8080，宿主机 Nginx 占 80/443）。

方案 B：阿里云 SSL 证书

在 [数字证书管理服务](#) 申请免费 DV 证书，下载 Nginx 格式，配置到宿主机或自定义 `frontend/nginx.conf` 后重建 `frontend` 镜像。

方案 C：阿里云 SLB

SLB 终结 HTTPS，后端 HTTP 转发到 ECS:80。

六、数据持久化与备份

Docker 命名卷（`docker-compose.prod.yml`）：

卷名	内容
postgres_data	文档元数据、会话、配置覆盖
minio_data	原始文件、表格/插图 PNG
qdrant_data	向量索引
elasticsearch_data	全文索引
model_cache	HuggingFace / 推理缓存

宿主机目录：

- `./models/` — Piper、BGE 模型（首次 bootstrap 下载，更新代码时勿删）

备份示例

```
# PostgreSQL
docker compose -f docker-compose.prod.yml exec -T postgres \
  pg_dump -U alldocs alldocs > backup_$(date +%F).sql

# 卷列表
docker volume ls | grep alldocs

# 打包 models（若需迁移）
tar czf models_backup.tar.gz -C /opt/alldocs models/
```

恢复：停服务 → 恢复卷/SQL → 再 `up -d`。

七、日常运维

7.1 常用命令

```
cd /opt/alldocs

# 重启全部
docker compose -f docker-compose.prod.yml up -d

# 仅重建 backend 相关 (代码更新后)
docker compose -f docker-compose.prod.yml up -d --build api worker-ingestion worker-mai

# 停止
docker compose -f docker-compose.prod.yml down

# 查看资源
docker stats
docker system df
```

7.2 日志与监控

- 日志: `docker compose -f docker-compose.prod.yml logs -f api worker-ingestion worker-maintenance`
- 指标: API `/metrics`、Inference `/metrics` 与 `/ready`、Worker `:9100` (内网, 见 [observability.md](#))

7.3 配置热更新

部分参数可通过前端设置面板或 `PATCH /api/v1/settings` 覆盖 `.env` (优先级更高)。改 `.env` 后通常需重启相关容器:

```
docker compose -f docker-compose.prod.yml restart api worker-ingestion worker-maintenan
```

八、故障排查

现象	可能原因	处理
无法访问 80	安全组未放行	控制台检查入方向 80
构建超时 / 拉镜像失败	Docker Hub 慢	确认 <code>/etc/docker/daemon.json</code> 镜像加速；重跑 <code>pull_docker_images.sh</code>
容器 exit 137	OOM	升配内存； <code>WHISPER_MODEL=small</code> ；确认 <code>INFERENCE_URL</code> 已设
API 连不上数据库	<code>.env</code> 仍用 <code>localhost</code>	按第三节改 Docker 服务名
入库一直 failed	ES/Qdrant/MinIO 未就绪	<code>docker compose logs worker-ingestion</code>
更新后配置丢失	上传时覆盖了 <code>.env</code>	重新编辑 <code>.env</code> ；同步时保持 <code>--exclude '.env'</code>
语音不可用	Piper 模型缺失	<code>bash scripts/download_piper_models.sh</code> <code>/opt/alldocs/models/piper</code>
登录后 401 / 无法访问 API	<code>AUTH_DISABLED</code> 与前端 token 不一致； <code>JWT_SECRET</code> 变更导致旧 token 失效	确认 <code>.env</code> 中 <code>AUTH_DISABLED=false</code> ；清除浏览器缓存重新登录
手机验证码收不到	阿里云 SMS 未配置或模板变量不匹配	检查 <code>SMS_PROVIDER=aliyun</code> 与 <code>ALIYUN_SMS_*</code> ；对照短信模板是否含 <code>code</code>
微信登录失败	回调域 / Redirect URI 与公网域名不一致	核对 <code>WECHAT_REDIRECT_URI</code> 、 <code>AUTH_FRONTEND_CALLBACK_URL</code> 与开放平台配置

进一步诊断：

```
docker compose -f docker-compose.prod.yml ps
docker compose -f docker-compose.prod.yml logs --tail=100 api worker-ingestion
dmesg | tail | grep -i oom
```

九、完整首次部署清单（Checklist）

- ☐ 创建 ECS (≥4核16G, Ubuntu 22.04, ≥100G 盘)
- ☐ 安全组: 22 (仅限办公 IP)、80 (公网)
- ☐ 开发机 rsync / 压缩包 上传代码到 /opt/alldocs
- ☐ `bash scripts/server-bootstrap.sh`
- ☐ `bash scripts/server-bootstrap.sh --generate-secrets`
- ☐ 编辑 `.env`: LLM_API_KEY、Redis/Qdrant/ES/MinIO/INFERENCE 内网地址
- ☐ 编辑 `.env`: JWT_SECRET、BOOTSTRAP_ADMIN_* (AUTH_DISABLED=false)
- ☐ (可选) 配置 ALIYUN_SMS_* 或 WECHAT_* 登录方式
- ☐ 备份 `.env` 到安全位置
- ☐ `bash scripts/server-bootstrap.sh --start`
- ☐ 浏览器访问 `http://<IP>`, 上传测试文档并问答
- ☐ (可选) 域名 + HTTPS

十、GitHub Actions 自动部署（可选）

若 ECS 已按第三节完成首次 bootstrap (`.env`、`models/` 就绪), 可启用 CI/CD:

1. 在 GitHub 仓库 **Settings** → **Secrets and variables** → **Actions** 配置:
2. `ECS_HOST`: ECS 公网 IP 或域名
3. `ECS_USER`: SSH 用户 (如 `root`)
4. `ECS_SSH_KEY`: 私钥全文 (含 `BEGIN ... PRIVATE KEY` 行)
5. 可选变量: `ECS_DEPLOY_PATH` (默认 `/opt/alldocs`)、`ECS_SSH_PORT` (默认 `22`)
6. 推送 `main` 或手动触发 **Deploy** workflow

Workflow 会先跑前后端测试, 再 `rsync` 同步代码 (排除 `.env`、`models/`), 在服务器执行 `scripts/remote_compose_deploy.sh start` 并轮询健康检查。`.env` 变更后仍需在 ECS 上手动编辑并重启相关容器。

十一、一键命令速查

开发机 — 首次上传 / 更新代码:

```
cd /path/to/AllDocs
rsync -avz --delete \
  --exclude '.git/' --exclude '.env' \
  --exclude 'node_modules/' --exclude 'frontend/node_modules/' \
  --exclude 'models/' \
  -e "ssh -i ~/.ssh/your-key.pem" \
  ./ root@<ECS公网IP>:/opt/alldocs/
```

ECS — 首次部署：

```
cd /opt/alldocs
bash scripts/server-bootstrap.sh
bash scripts/server-bootstrap.sh --generate-secrets
vim .env # LLM_API_KEY + Docker 内网 URL + JWT_SECRET + BOOTSTRAP_ADMIN_*
bash scripts/server-bootstrap.sh --start
```

ECS — 版本更新后重建：

```
cd /opt/alldocs
bash scripts/pull_docker_images.sh
docker compose -f docker-compose.prod.yml up -d --build
```

ECS — 健康检查：

```
curl -s http://127.0.0.1/ -o /dev/null -w "%{http_code}\n"
docker compose -f docker-compose.prod.yml exec -T api \
  python -c "import urllib.request; urllib.request.urlopen('http://localhost:8000/health')
```